

CAPSTONE PROJECT

Name – Sristi Saha

Institute – Institute of Engineering & Management, Kolkata

Enrollment No. – 12023052016066

Project Number Selected	2
Project Topic	E-Commerce Login and Product Search Automation
Application Under Test	TutorialsNinja OpenCart Demo Store
Application URL	https://tutorialsninja.com/demo/
Technology Stack	Python, Selenium WebDriver, pytest, unittest, CSV, INI, pytest-html

1. Objective

The objective of this capstone project is to design and implement a reusable, maintainable Selenium Python automation framework for validating essential e-commerce user journeys. The framework automates login validation and product search on the TutorialsNinja demo application, while demonstrating both unittest and pytest, the Page Object Model (POM), utility classes, configuration management, CSV-based test data, failure screenshots, and HTML test reporting.

- Create a modular framework that separates test logic, page interactions, configuration, and test data.
- Automate negative login validation and catalogue search scenarios.
- Support execution through both pytest and Python unittest.
- Use explicit waits and centralized page objects to improve reliability and maintainability.
- Capture screenshots when tests fail and produce an HTML report for pytest execution.

2. Project Structure

The project is organized into configuration, framework utilities, page objects, test data, tests, and output directories.

```
capstone_project_selenium_ecommerce_framework/
```

```
├── config/
│   └── config.ini
├── framework/
│   ├── base_page.py
│   ├── config_reader.py
│   ├── data_reader.py
│   ├── driver_factory.py
│   └── screenshot_util.py
├── pages/
│   ├── home_page.py
│   ├── login_page.py
│   └── search_results_page.py
├── testdata/
│   ├── login_data.csv
│   └── search_data.csv
├── tests/
│   ├── pytest/
│   │   ├── conftest.py
│   │   ├── test_login.py
│   │   └── test_product_search.py
│   └── unittest/
│       └── test_login_unittest.py
├── reports/
├── screenshots/
├── pytest.ini
├── requirements.txt
└── README.md
```

3. Methodology

1. **Requirement analysis:** Identify the target application and the two core workflows: login validation and product search.
2. **Framework design:** Separate test cases from page-specific interactions using POM and a shared BasePage.
3. **Configuration management:** Read URL, browser, headless mode, timeouts, and recording pause from config/config.ini.
4. **Test data preparation:** Store synthetic invalid login combinations and search keywords in CSV files, then load them through CSVDataReader.
5. **Browser initialization:** Create a fresh browser session for each test through DriverFactory; Selenium Manager can resolve drivers in supported environments.
6. **Test execution:** Use pytest parameterization for CSV-driven cases and unittest for an independent login validation case.
7. **Synchronization and assertions:** Use explicit waits for visibility and clickability, then assert warning visibility or matching search results.
8. **Failure evidence and reporting:** Capture screenshots on failure; pytest-html produces a standalone HTML report with failure image attachments where supported.
9. **Execution review:** Run pytest and unittest separately, inspect console results, review the report, and retain screenshots as submission evidence.

4. Test Scenarios

Test ID	Framework	Scenario	Expected outcome
LGN-001	pytest + CSV	Submit invalid credentials	Login is rejected and a warning is displayed.
LGN-002	unittest	Submit invalid credentials	A non-empty login validation warning is displayed.
SRCH-001	pytest + CSV	Search for iPhone	Search heading and at least one matching product appear.
SRCH-002	pytest + CSV	Search for MacBook	Search heading and at least one matching product appear.

5. Source Codes

requirements.txt

```
selenium>=4.20,<5
pytest>=8.0,<9
pytest-html>=4.1,<5
```

pytest.ini

```
[pytest]
testpaths = tests/pytest
python_files = test_*.py
python_classes = Test*
```

```
python_functions = test_*
addopts = -ra
```

config/config.ini

```
[application]
base_url = https://tutorialsninja.com/demo/
browser = chrome
headless = false
explicit_wait = 15
page_load_timeout = 30

[reporting]
screenshot_dir = screenshots
report_dir = reports
```

framework/config_reader.py

```
from configparser import ConfigParser
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
CONFIG_PATH = PROJECT_ROOT / "config" / "config.ini"

class ConfigReader:
    def __init__(self, path=CONFIG_PATH):
        self._config = ConfigParser()
        loaded = self._config.read(path, encoding="utf-8")
        if not loaded:
            raise FileNotFoundError(f"Configuration file not found: {path}")

    def get(self, section, key, fallback=None):
        return self._config.get(section, key, fallback=fallback)

    def get_bool(self, section, key, fallback=False):
        return self._config.getboolean(section, key, fallback=fallback)

    def get_int(self, section, key, fallback=0):
        return self._config.getint(section, key, fallback=fallback)
```

framework/driver_factory.py

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options as ChromeOptions
from selenium.webdriver.firefox.options import Options as FirefoxOptions
from framework.config_reader import ConfigReader

class DriverFactory:
    @staticmethod
    def create_driver():
        config = ConfigReader()
```

```

browser = config.get("application", "browser", "chrome").strip().lower()
headless = config.get_bool("application", "headless", False)

if browser == "chrome":
    options = ChromeOptions()
    if headless:
        options.add_argument("--headless=new")
        options.add_argument("--window-size=1440,1000")
        options.add_argument("--disable-notifications")
        options.add_argument("--disable-popup-blocking")
        driver = webdriver.Chrome(options=options)
elif browser == "firefox":
    options = FirefoxOptions()
    if headless:
        options.add_argument("-headless")
        driver = webdriver.Firefox(options=options)
        driver.set_window_size(1440, 1000)
else:
    raise ValueError(f"Unsupported browser '{browser}'. Use chrome or firefox.")

driver.set_page_load_timeout(config.get_int("application", "page_load_timeout", 30))
return driver

```

framework/data_reader.py

```

import csv
from pathlib import Path
from framework.config_reader import PROJECT_ROOT

class CSVDataReader:
    @staticmethod
    def read_rows(filename):
        path = PROJECT_ROOT / "testdata" / filename
        with path.open(mode="r", newline="", encoding="utf-8-sig") as csv_file:
            return list(csv.DictReader(csv_file))

    @staticmethod
    def read_column(filename, column):
        return [row[column] for row in CSVDataReader.read_rows(filename)]

```

framework/screenshot_util.py

```

from datetime import datetime
from pathlib import Path
from framework.config_reader import ConfigReader, PROJECT_ROOT

class ScreenshotUtil:
    @staticmethod
    def capture(driver, test_name):
        config = ConfigReader()

```

```

directory = PROJECT_ROOT / config.get("reporting", "screenshot_dir", "screenshots")
directory.mkdir(parents=True, exist_ok=True)
safe_name = "".join(c if c.isalnum() or c in "-_" else "_" for c in test_name)
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
path = directory / f"{safe_name}_{timestamp}.png"
if driver:
    driver.save_screenshot(str(path))
    return str(path)
return None

```

framework/base_page.py

```

from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from framework.config_reader import ConfigReader

class BasePage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(
            driver, ConfigReader().get_int("application", "explicit_wait", 15)
        )

    def click(self, locator):
        self.wait.until(EC.element_to_be_clickable(locator)).click()

    def type_text(self, locator, text, clear=True):
        element = self.wait.until(EC.visibility_of_element_located(locator))
        if clear:
            element.clear()
        element.send_keys(text)
        return element

    def get_text(self, locator):
        return self.wait.until(EC.visibility_of_element_located(locator)).text

    def is_visible(self, locator):
        try:
            return self.wait.until(EC.visibility_of_element_located(locator)).is_displayed()
        except Exception:
            return False

```

pages/home_page.py

```

from selenium.webdriver.common.by import By
from framework.base_page import BasePage

class HomePage(BasePage):
    SEARCH_INPUT = (By.NAME, "search")
    SEARCH_BUTTON = (By.CSS_SELECTOR, "#search button")

```

```

def open(self, base_url):
    self.driver.get(base_url)
    self.wait.until(lambda d: d.title != "")
    return self

def search_for(self, keyword):
    self.type_text(self.SEARCH_INPUT, keyword)
    self.click(self.SEARCH_BUTTON)
    return SearchResultsPage(self.driver)

```

```

from pages.search_results_page import SearchResultsPage

```

pages/login_page.py

```

from selenium.webdriver.common.by import By
from framework.base_page import BasePage

class LoginPage(BasePage):
    EMAIL = (By.ID, "input-email")
    PASSWORD = (By.ID, "input-password")
    LOGIN_BUTTON = (By.CSS_SELECTOR, "input[type='submit'][value='Login']")
    WARNING = (By.CSS_SELECTOR, ".alert.alert-danger")

    def open(self, base_url):
        self.driver.get(base_url.rstrip("/") + "/index.php?route=account/login")
        self.wait.until(lambda d: "login" in d.current_url.lower())
        return self

    def login(self, email, password):
        self.type_text(self.EMAIL, email)
        self.type_text(self.PASSWORD, password)
        self.click(self.LOGIN_BUTTON)
        return self

    def warning_text(self):
        return self.get_text(self.WARNING)

```

pages/search_results_page.py

```

from selenium.webdriver.common.by import By
from framework.base_page import BasePage

class SearchResultsPage(BasePage):
    HEADING = (By.CSS_SELECTOR, "#content h1")
    PRODUCT_CARDS = (By.CSS_SELECTOR, "div.product-thumb")
    EMPTY_MESSAGE = (By.XPATH, "//*[contains(normalize-space(), 'There is no product that matches')]")

    def heading(self):

```

```

    return self.get_text(self.Heading)

def product_count(self):
    self.wait.until(lambda d: d.find_elements(*self.PRODUCT_CARDS) or
                     d.find_elements(*self.EMPTY_MESSAGE))
    return len(self.driver.find_elements(*self.PRODUCT_CARDS))

def product_names(self):
    cards = self.driver.find_elements(*self.PRODUCT_CARDS)
    names = []
    for card in cards:
        title = card.find_elements(By.CSS_SELECTOR, ".caption h4 a")
        if title:
            names.append(title[0].text.strip())
    return names

```

tests/pytest/conftest.py

```

import pytest
from framework.config_reader import ConfigReader
from framework.driver_factory import DriverFactory
from framework.screenshot_util import ScreenshotUtil

@pytest.fixture
def driver(request):
    web_driver = DriverFactory.create_driver()
    request.node._selenium_driver = web_driver
    yield web_driver
    web_driver.quit()

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    outcome = yield
    report = outcome.get_result()
    if report.when == "call" and report.failed:
        web_driver = getattr(item, "_selenium_driver", None)
        if web_driver:
            path = ScreenshotUtil.capture(web_driver, item.name)
            if path:
                extras = getattr(report, "extras", [])
                try:
                    import pytest_html
                    extras.append(pytest_html.extras.image(path, name="Failure screenshot"))
                except Exception:
                    pass
                report.extras = extras

```

tests/pytest/test_login.py

```
import pytest
from framework.config_reader import ConfigReader
from framework.data_reader import CSVDataReader
from pages.login_page import LoginPage

@pytest.mark.parametrize("row", CSVDataReader.read_rows("login_data.csv"))
def test_invalid_login_shows_validation_message(driver, row):
    config = ConfigReader()
    page = LoginPage(driver).open(config.get("application", "base_url"))
    page.login(row["email"], row["password"])
    assert page.is_visible(page.WARNING), "Expected an invalid-login warning to be displayed."
    assert "warning" in page.warning_text().lower() or "no match" in page.warning_text().lower()
```

tests/pytest/test_product_search.py

```
import pytest
from framework.config_reader import ConfigReader
from framework.data_reader import CSVDataReader
from pages.home_page import HomePage

@pytest.mark.parametrize("row", CSVDataReader.read_rows("search_data.csv"))
def test_product_search_returns_matching_results(driver, row):
    config = ConfigReader()
    results = HomePage(driver).open(config.get("application", "base_url")).search_for(row["keyword"])
    assert row["expected_fragment"].lower() in results.heading().lower()
    assert results.product_count() > 0, f"No products returned for '{row['keyword']}'."
    assert any(row["expected_fragment"].lower() in name.lower() for name in results.product_names()), (
        f"No displayed product title matched '{row['expected_fragment']}'."
    )
```

tests/unittest/test_login_unittest.py

```
import unittest
from framework.config_reader import ConfigReader
from framework.driver_factory import DriverFactory
from framework.screenshot_util import ScreenshotUtil
from pages.login_page import LoginPage

class LoginUnittest(unittest.TestCase):
    def setUp(self):
        self.config = ConfigReader()
        self.driver = DriverFactory.create_driver()

    def tearDown(self):
        if hasattr(self, "_outcome") and self._outcome.errors:
            for _, error in self._outcome.errors:
                if error:
                    ScreenshotUtil.capture(self.driver, self.id().split(".")[-1])
                    break
```

```
self.driver.quit()
```

```
def test_invalid_login_displays_warning(self):
```

```
    page = LoginPage(self.driver).open(self.config.get("application", "base_url"))
```

```
    page.login("invalid@example.com", "IncorrectPassword123")
```

```
    self.assertTrue(page.is_visible(page.WARNING), "Invalid login warning was not visible.")
```

```
    self.assertTrue(page.warning_text().strip())
```

```
if __name__ == "__main__":
```

```
    unittest.main(verbosity=2)
```

testdata/login_data.csv

```
email,password,scenario
```

```
invalid@example.com,WrongPassword123,invalid_credentials
```

```
not-an-email,SomePassword123,invalid_email_format
```

testdata/search_data.csv

```
keyword,expected_fragment,scenario
```

```
iphone,iphone,common_product
```

```
macbook,macbook,common_product
```

6. Execution Commands

```
# Install dependencies
```

```
python -m pip install -r requirements.txt
```

```
# Run pytest with verbose output and standalone HTML report
```

```
pytest tests/pytest -v --html=reports/pytest-report.html --self-contained-html
```

```
# Run unittest independently
```

```
python -m unittest discover -s tests/unittest -p "test_*.py" -v
```

7. Output

Figure 1. Customer Login

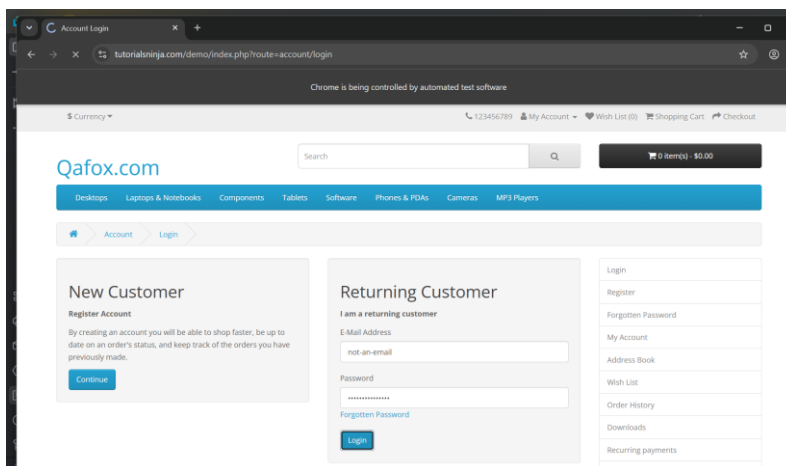


Figure 2. MacBook product search results

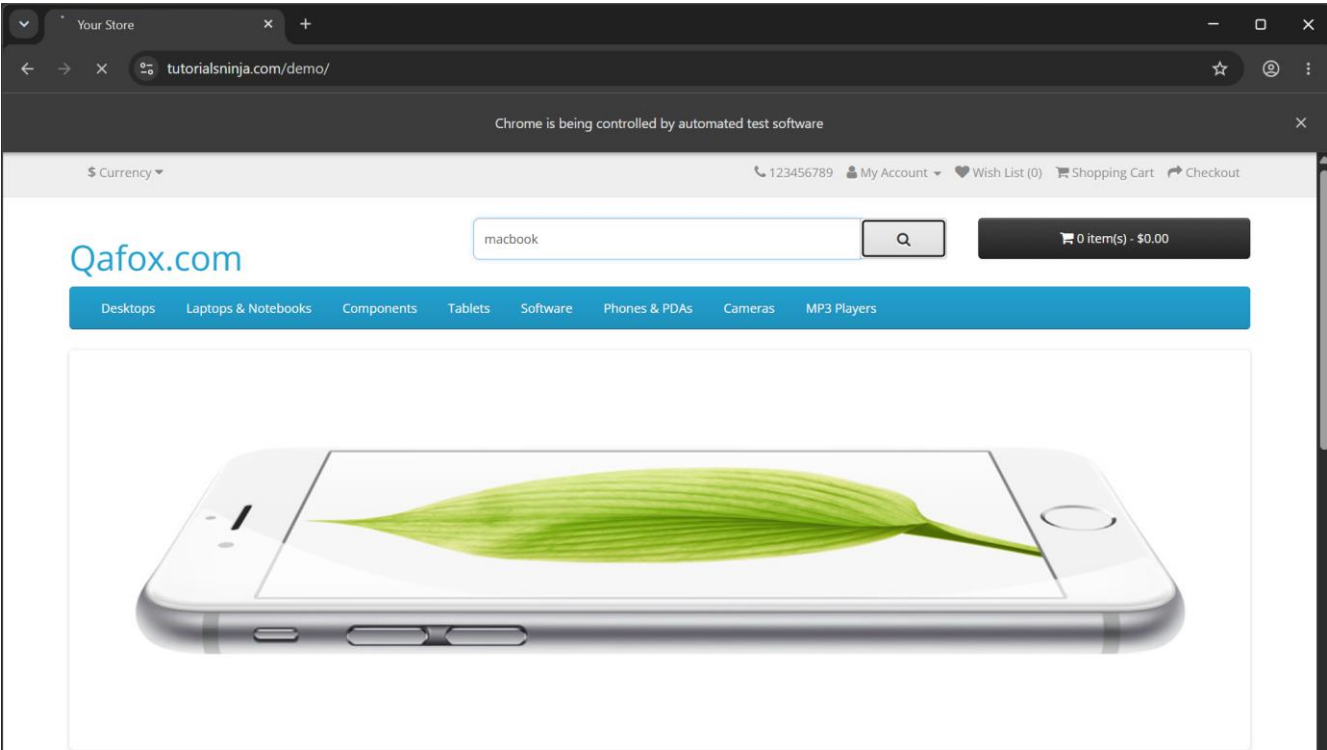


Figure 3. HTML report

pytest-report.html

Report generated on 28-Sep-2026 at 02:07:43 by [pytest-html](#) v4.2.0

Environment

Python	3.12.4
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 8.4.2 • pluggy: 1.6.0
Plugins	• anyio: 4.10.0 • html: 4.2.0 • metadata: 3.1.1
JAVA_HOME	C:\Program Files\Java\jdk-25

Summary

4 tests took 00:00:34.

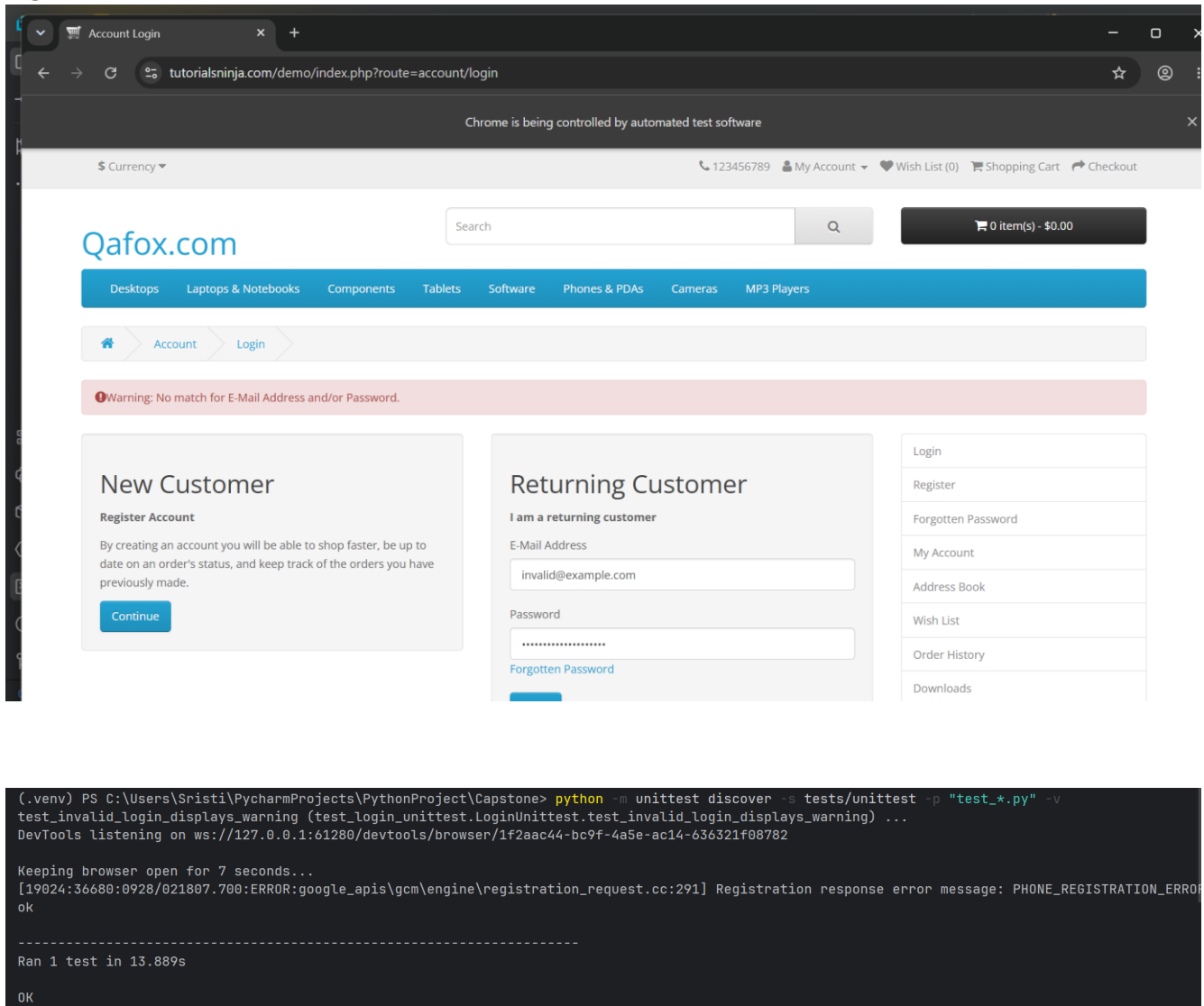
(Un)check the boxes to filter the results.

☒ 0 Failed, ☒ 4 Passed, ☒ 0 Skipped, ☒ 0 Expected failures, ☒ 0 Unexpected passes, ☒ 0 Errors, ☒ 0 Reruns ☒ 0 Retried,

[Show all details](#) / [Hide all details](#)

Result	Test	Duration	Links
Passed	tests/pytest/test_login.py::test_invalid_login_shows_validation_message[row0]	00:00:08	
Passed	tests/pytest/test_login.py::test_invalid_login_shows_validation_message[row1]	00:00:10	
Passed	tests/pytest/test_product_search.py::test_product_search_returns_matching_results[row0]	00:00:08	
Passed	tests/pytest/test_product_search.py::test_product_search_returns_matching_results[row1]	00:00:08	

Figure 4. unittest execution



8. Results and Observation

During the recorded pytest execution, the four parameterized test cases completed successfully with the console summary: "4 passed in 34.21s". This indicates that the two invalid-login data cases and the two product-search cases passed in that run. The subsequent unittest run exposed a teardown compatibility issue: the implementation attempted to access the private unittest `_Outcome.errors` attribute, which was not available in the execution environment. The test therefore reported an error during cleanup; this is distinct from a demonstrated failure of the login assertion itself.

The teardown implementation was revised to avoid relying on the private `_Outcome.errors` attribute and to capture a screenshot when an exception occurs. A configurable seven-second browser pause was also requested for screen recording. The final corrected unittest result should be recorded only after rerunning it in the target environment.

- POM improves maintainability by centralizing page locators and user actions.
- CSV parameterization allows test inputs to be extended without duplicating test methods.
- Explicit waits reduce dependence on fixed delays during element interactions.

- Separate pytest and unittest suites demonstrate framework interoperability while allowing independent execution.
- The demo website and its catalogue are external dependencies; content or availability changes may require test-data updates.

9. Conclusion

The capstone project implements a modular Selenium Python automation framework for an e-commerce demo application. It brings together pytest, unittest, Page Object Model, reusable utilities, INI configuration, CSV-driven test data, failure screenshots, and HTML reporting. The recorded pytest run passed all four test cases. A unittest cleanup compatibility issue was identified and the teardown logic was updated; the corrected unittest suite must be rerun to confirm its final status. Overall, the framework provides a practical foundation for extending UI automation coverage, improving regression repeatability, and producing reviewable execution evidence.